From 4D to the Web

HTML - CSS - JAVASCRIPT
For ~~dummies~~ beginners

4D Form

Adagio

00:16 02:01

STYLES

- All (11 styles)
- • Brésil/Portugal
- Celtique
- Chanson francophone
- Classique
- Divers
- Jazz
- Latin
- Musique de film
- New age
- Pop/Rock

ARTISTS

- All (32 artists)
- • Antônio Carlos Jobim
- Antonion Carlos Jobim + Sting
- Astrud Gilberto
- Baden Powell
- Bebel Gilberto
- Bevinda
- Cesaria Evora
- Chico Buarque
- Compay Segundo
- Cristina Branco

ALBUMS

ANTONIO BRASILEIRO

- • Só Danço Samba
- Piano Na Mangueira
- How Insensitive (Insensatez)
- Querida
- Surfboard
- Samba De Maria Luiza
- Forever Green
- Maracangalha
- Maricotinha
- Pato Preto
- Meu Amigo Radamés
- Blue Train (Trem Azul)
- Radamés Y Pelé
- Chora Coração
- Trem De Ferro

Web Form

Adagio
00:00
00:00
Styles
All styles
Brésil/Portugal
Celtique
Chanson francophone
Classique
Divers
Jazz
Latin
Musique de film
New age
Pop/Rock
World
Artists
All artists
Antônio Carlos Jobim
Antonion Carlos Jobim + Sting
Astrud Gilberto
Baden Powell
Bebel Gilberto
Bevinda
Cesaria Evora
Chico Buarque
Compay Segundo
Cristina Branco
Divers Portugal
Gilberto Gil
Gilberto Gil & Milton Nascimento
Albums
Antonio Brasileiro
JOBIM
ANTONIO BRASILEIRO
Tracks
S Dano Samba
Piano Na Mangueira
How Insensitive (Insensatez)
Querida
Surfboard
Samba De Maria Luiza
Forever Green
Maracangalha
Maricotinha
Pato Preto
Meu Amigo Radams
Blue Train (Trem Azul)
Radams Y Pel
Chora Corao
Trem De Ferro

HTML - CSS - JAVASCRIPT

- **HTML**: Language used to display *raw* data.
- **CSS**: Language used to improve the presentation
- **Javascript**: Language used to do… everything else!

HTML: Main tags

Main tag: <html>	This is the main tag, that starts and ends a web page
Head tag: <head>	This tag will hold info about the content of the web page
Body tag: <body>	This tag will contain the data to display

HTML: Sub tags

Head tags:

<title>Adagio</title>

<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">

Body tags:

Static texts: <h1>, <h2>, <h3>, <h4>, <p>

Lists: <ul>, <li>

Tables: <table>, <tr>, <td>, <th>, <thead>, <tfoot>, <tbody>

Buttons: <button>

Forms: <form>

Input zone: <input>

Images: <img>

Links: <a>

…

HTML - Attributes

Inside each tag you may find one or more **attributes…**

```
<input id="cb1" type="checkbox" value="Chocolate">
<img src="/Adagio_files/Cover_142">
```

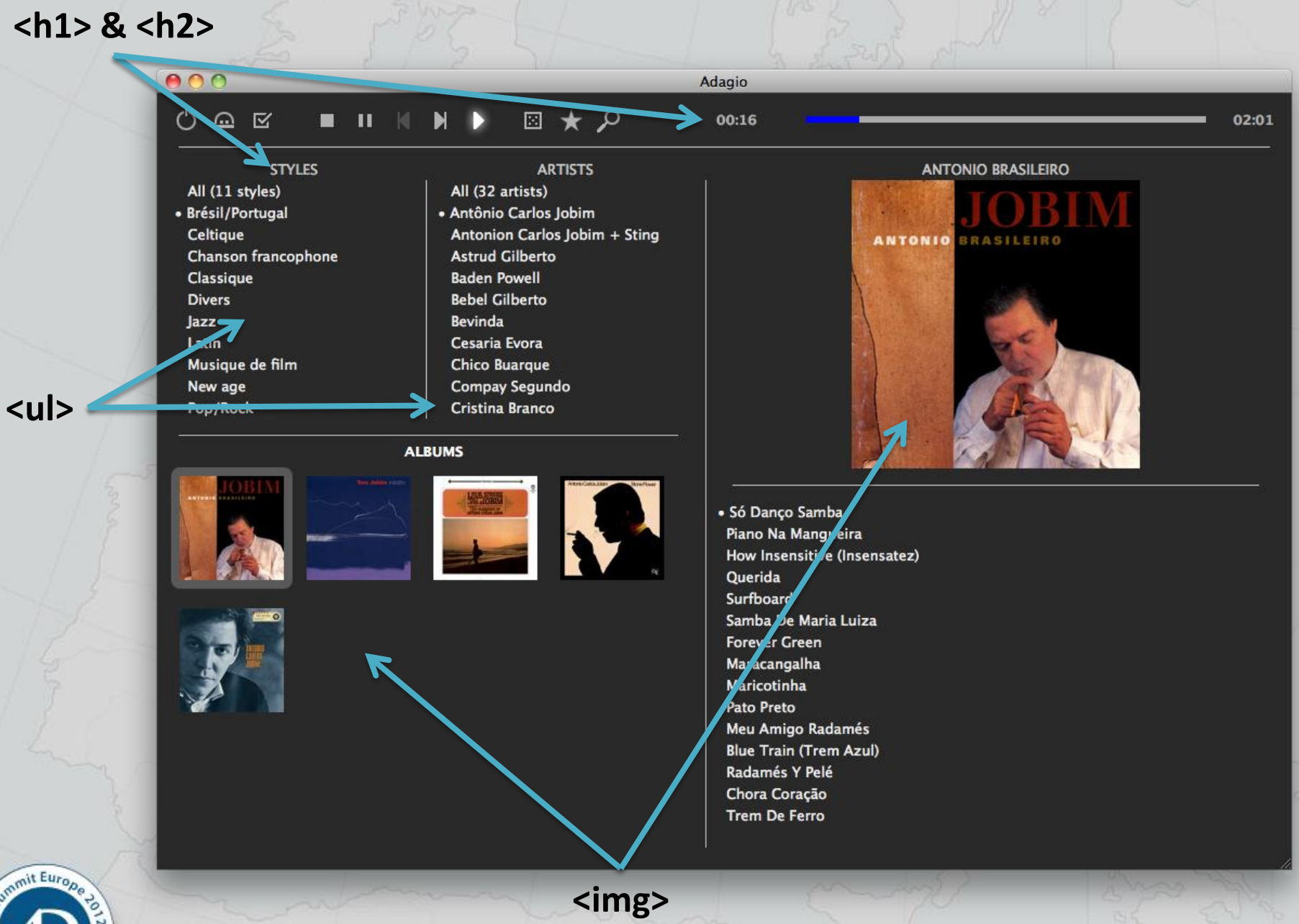
HTML – In this session…

Static texts: Huge headers (<**h1**>) and Big headers(<**h2**>)

Unordered Lists: (**<ul>**) and **lines** (**<li>**)

Images: **<img>**

That's (almost) it!


<h1> & <h2>
Adagio
00:16
02:01
STYLES
All (11 styles)
• Brésil/Portugal
Celtique
Chanson francophone
Classique
Divers
Jazz
Latin
Musique de film
New age
Pop/Rock
ARTISTS
All (32 artists)
• Antônio Carlos Jobim
Antonion Carlos Jobim + Sting
Astrud Gilberto
Baden Powell
Bebel Gilberto
Bevinda
Cesaria Evora
Chico Buarque
Compay Segundo
Cristina Branco
ALBUMS
ANTONIO BRASILEIRO
JOBIM
ANTONIO BRASILEIRO
• Só Danço Samba
Piano Na Mangueira
How Insensitive (Insensatez)
Querida
Surfboard
Samba De Maria Luiza
Forever Green
Marracangalha
Maricotinha
Pato Preto
Meu Amigo Radamés
Blue Train (Trem Azul)
Radamés Y Pelé
Chora Coração
Trem De Ferro
<ul>
<img>

HTML: Raw Data

Demo

CSS: Cascading Style Sheets

Rules:

#1 As much as possible, HTML should NOT define any style, position or whatever.

#2 CSS will be used to define the font sizes, colors, position of objects, behaviors, etc.

#3 All the "graphics" and "effects" should be managed within CSS files.

“Anything that can be done using CSS must be done using CSS”

CSS: Cascading Style Sheets

Hierarchy: Every object inherits from its parents

References: HTML objects can be referenced using their **type** (html tags), their "**id**"s or their "**classes**"

Examples:

- All the headers (<h1>) should be Red and **Bold**
- All the lists (<ul>) should have vertical scrollbars
- The object whose **id**="alpha" should be located on the bottom right

CSS: Cascading Style Sheets

Example #1: How should <h1> texts be displayed?

HTML

```
<h1>Artists</h1>
```

CSS

```
h1 {
    font-family:verdana,arial,sans-serif;
    font-size:13px;
    font-style:normal;
    text-align:center;
    color:white;
    }
```

CSS: Cascading Style Sheets

Example #2: How should <h1> texts *located inside a whose id is "navigation"* be displayed?

HTML

```
<div id = "navigation">
    <h1>Styles</h1)
</div>
```

CSS

```
#navigation h1
{
color: #d0d0d0;
font-size:14px;
font-weight: bold;
}
```

CSS: Cascading Style Sheets

Example #3: How should <**img**> images *located inside a whose* ***id*** *is "ulAlbums"* with the **class** "Selected" be displayed?

HTML

```
<div id="ulAlbums">
    <img src="/Cover_142"><img class="selected" src="/Cover_143">
</div>
```

CSS

```
#ulAlbums img.selected
{
margin:3px;
opacity:1;
border-width:3px;
border-style:solid;
border-color: #0080F0;
}
```

CSS: Cascading Style Sheets

Example #4: Pseudo classes

Some classes are *predefined* like "hover", "link", "visited", "active"…

They are described using a ":" rather than a "."

CSS

```
#ulAlbums:hover
{
opacity:0.8;
}
```

CSS: Cascading Style Sheets

Demo(s)

Javascript: The Web language

- The language is "object" and based on "events"
- Functions can be associated with objects and events.
- Synchronous / Asynchronous

Javascript: The Web language

Simple example: associate a function with a list when the list is clicked

HTML + Javascript

```
<ul id="ulStyles" onclick ="listClicked(this, event)">
```

4D Equivalent ("ulStyles" object method)

```
if(Form event=[illegible])
    listClicked(Self;Form event)
End if
```

Javascript: The Web language

Better example: do not mix HTML and Javascript.

HTML

```
<body onload="loadPlayer()">
```

Javascript

```
function loadPlayer()
        {
    myObject=document.getElementsById('ulStyles');
    myObject.addEventListener('click', function(event)
                {
                  listClicked(this, event);
                });
    }
```

Javascript: The Web language

Even better example: use classes!

HTML

```
<ul id="ulStyles" class="listenClicks">…
<ul id="ulArtists" class="listenClicks">…
<div id ="ulAlbums" class="listenClicks">…
<ul id="ulTracks" class="listenClicks">…
```

Javascript

```
function loadPlayer()
    {
        var list = document.getElementsByClassName('listenClicks');

            for ( i = 0; i < list.length; i += 1)
            {
                list[i].addEventListener('click', function(event)
                {
                    listClicked(this, event);
                });
            }
    }
```

Javascript: The Web language

Let's use Javascript to change a class dynamically!

HTML

```
<ul id="ulStyles" class="listenClicks">
    <li id="style_5" class="">Classical</li>
    <li id="style_7" class="Selected">Jazz</li>
        (...)
```

Javascript

```
function listClicked(which,what) (which is the <ul>, not the <li>!)
{
var n = which.childElementCount;
    for(i=0;i<n;i++)
    {which.children[i].className="";}

what.target.className="selected";
}
```

Javascript: The Web language

Demo

(Introducing M.C. Debugger! ;o)

#2: onClick
Call4D

#1: onLoad
Call4D

#3: onClick
Call4D

#4: onClick
Call4D

Javascript: Let's call 4D!

How can 4D interact with the HTML Content?

Example #1: fill the <ul> whose id="ulStyles".

```
<ul id="ulStyles">
    <li id="style_1" class="Selected">Classical</li>
    <li id="style_2" class="x">Jazz</li>
    <li id="style_3" class="x">Latin</li>
    <li id="style_4" class="x">New age</li>
</ul>
```

Javascript: Let's call 4D!

Javascript:

```
document.getElementById("ulStyles").innerHTML="Something"
```

HTML:

```
<ul id="ulStyles">
        Something
</ul>
```

Javascript: Let's call 4D!

Javascript:

```
req = new XMLHttpRequest(); (…)

req.open("GET","url4D",true); (…)

req.onreadystatechange=function() (…)

req.responseText (…)
```

Javascript: Let's call 4D!

"The Ugly, the Bad and the Good"

The Ugly:	what you should **never** do
The (not so) Bad:	what you *may* do
The Good:	what you <u>should</u> do

Javascript: Let's call 4D!

"The Ugly…"

Build HTML within a 4D Method

```
$HTML:="<li>jazz</li>"+"<li>classical</li>"+…
Web send text ($HTML)
```

☹ Yuck ☹

Javascript: Let's call 4D!

"The (not so) bad..."

Send a .SHTML file

```
<!--#4DLOOP _StylesNames-->
<li id="style_<!--#4DTEXT _StylesNames-->"><!--
   #4DTEXT _StylesNames{_StylesNames}--></li>
<!--#4DENDLOOP -->
```

😐 😐 😐

Javascript: Let's call 4D!

"The GOOD!"

Send JSON text!

```
{
"attribute1":"value1",
"attribute2":"value2",
"attribute3":"value3"
}
```

☺ ☺ ☺

Javascript: Let's call 4D!

"The GOOD!"

Send JSON text!


```
{
"arrStyles":[
{"id":"style_1","klass":"x","name":"All styles"},
{"id":"style_5","klass":"x","name":"Classical"},
{"id":"style_12","klass":"x","name":"World"},]
}
```

Javascript: Let's call 4D!

"The very GOOD!"

Ultimate trick: Build JSON text within a .shtml file!

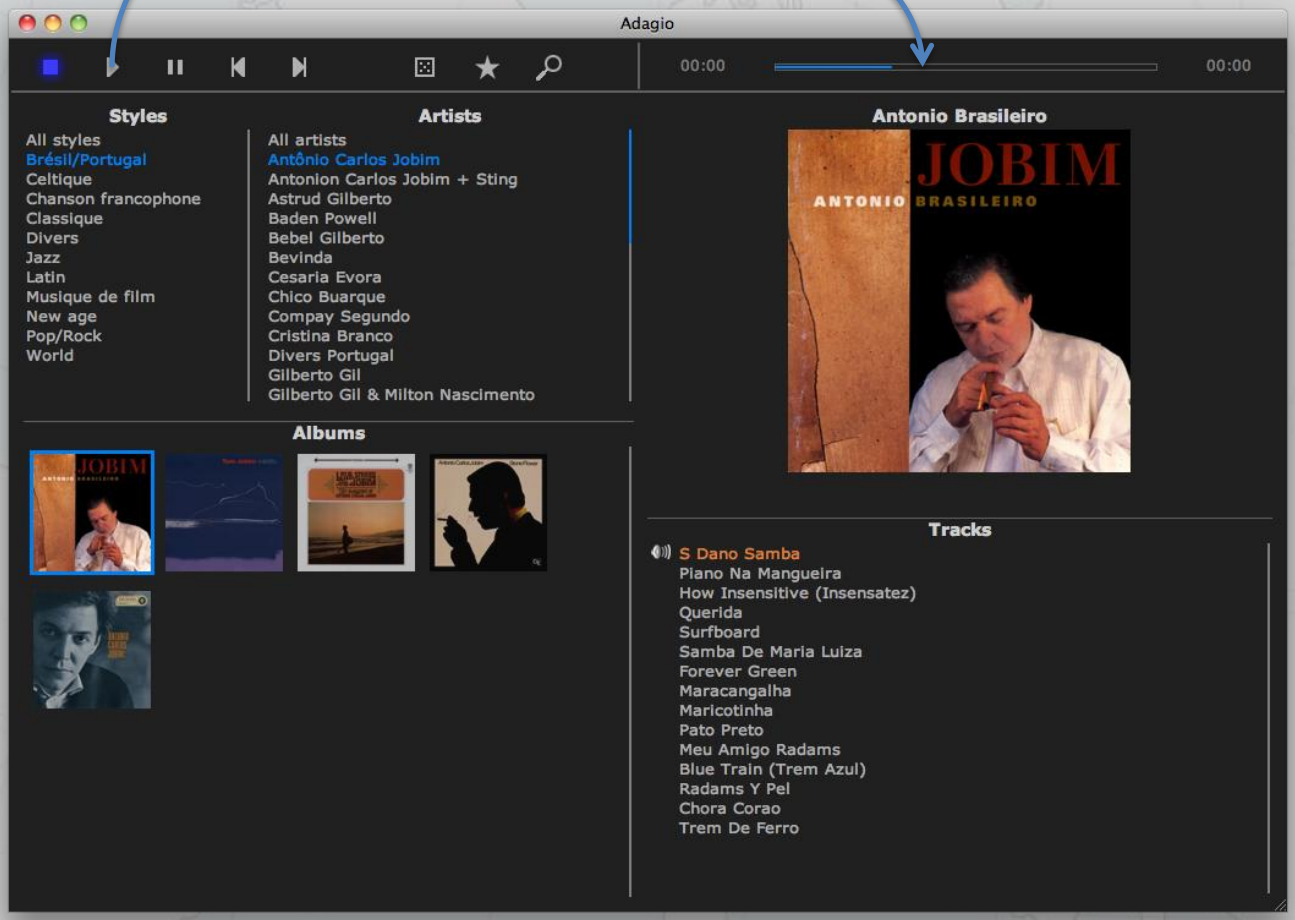
```
{
"arrStyles": [<!--#4DLOOP _StylesNames-->
{"id":"style_<!--#4DTEXT _StylesNames-->",
"klass":"Selected",
"name":"<!--#4DTEXT _StylesNames{_StylesNames}-->"},
<!--#4DENDLOOP -->]
}
```

Javascript: Let's call 4D!

Demo!

#5

onClick: **Play!**



One more thing…

…let's "Think different"!